```
import os
import sys
import cv2
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure
IMAGE_PATH = "/workspaces/computer-vision-Experiments./EXPERIMENT_05 _IMAGE.jpg"
OUTPUT_DIR = "output"
os.makedirs(OUTPUT_DIR, exist_ok=True)
def save_figure(filename, title=None, figsize=(8, 8)):
    if title is not None:
        plt.title(title)
        plt.axis("off")
output_path = os.path.join(OUTPUT_DIR, filename)
plt.savefig(
        output_path,
        bbox_inches="tight",
        dpi=200 )
plt.close()
print(f"[SAVED] {output_path}")
print("=" * 70)
print("EXPERIMENT NO. 5")
print("Feature Extraction and Image Analysis using SIFT and HOG")
print("=" * 70)
print("\n[STEP 1] Loading image...")
img = cv2.imread(IMAGE_PATH)
if img is None:
    print("\nERROR: Image could not be loaded.")
    print(f"Make sure '{IMAGE_PATH}' exists in the project folder.")
    sys.exit(1)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
height, width = img.shape[:2]
```

```
print("Image loaded successfully.")
print(f"Image width  : {width}")
print(f"Image height : {height}")
print(f"Image channels: {img.shape[2]}")
plt.figure(figsize=(8, 10))
plt.imshow(img_rgb)
save_figure(
    "01_original_image.png",
    "Original Image",
    figsize=(8, 10))
```




```
print("\n[STEP 2] Converting image to grayscale...")
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
print("Grayscale conversion completed.")
plt.figure(figsize=(8, 10))
plt.imshow(gray,cmap="gray")
save_figure( "02_grayscale_image.png","Grayscale Image",figsize=(8, 10))
```



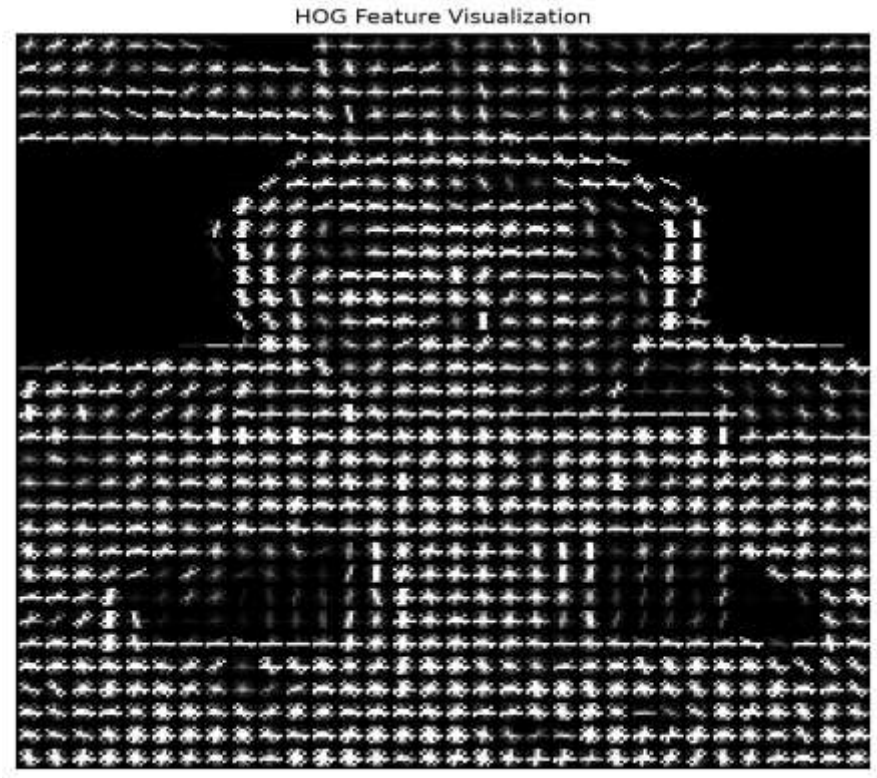

```
print("\n[STEP 3] Applying SIFT...")
try:
    sift = cv2.SIFT_create()
except AttributeError:
    print("\nERROR: SIFT is not available.")
    print("Install opencv-contrib-python using:")
    print("pip install opencv-contrib-python")
    sys.exit(1)
keypoints, descriptors = sift.detectAndCompute(gray,None)
print("\n" + "-" * 50)
print("SIFT RESULTS")
print("-" * 50)
print("Number of keypoints:", len(keypoints))
if descriptors is not None:
    print("Descriptor shape:", descriptors.shape)
else:
    print("No descriptors detected.")
print("\n[STEP 4] Visualizing SIFT keypoints...")
sift_image = cv2.drawKeypoints(img_rgb,
keypoints,None,flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
plt.figure(figsize=(8, 10))
plt.imshow(sift_image)
```

```
save_figure(
    "03_sift_keypoints.png",
    "SIFT Keypoints",
    figsize=(8, 10))
```




```
print("\n[STEP 5] Extracting HOG features...")
hog_input = cv2.resize(gray,(256, 256))
ORIENTATIONS = 9
PIXELS_PER_CELL = (
    8, 8) CELLS_PER_BLOCK = (2,2)
hog_features, hog_image = hog(
    hog_input,
    orientations=ORIENTATIONS,
    pixels_per_cell=PIXELS_PER_CELL,
    cells_per_block=CELLS_PER_BLOCK,
    block_norm="L2-Hys",
    visualize=True)
print("\n" + "-" * 50)
print("HOG RESULTS")
print("-" * 50)
print("Orientations:", ORIENTATIONS)
```

```
print( "Pixels per cell:",PIXELS_PER_CELL)
print( "Cells per block:",CELLS_PER_BLOCK)
print("HOG feature vector length:",len(hog_features))
print("\n[STEP 6] Creating HOG visualization...")
hog_visual = exposure.rescale_intensity(
   hog_image,
   in_range=(0, 10))
plt.figure(figsize=(8, 8))
plt.imshow(
    hog_visual,
    cmap="gray")
save_figure("04_hog_visualization.png","HOG Feature Visualization",
    figsize=(8, 8))
```




```
print("\n[STEP 7] Creating second similar image...")
center = (
    width // 2,
    height // 2
)rotation_matrix = cv2.getRotationMatrix2D(
    center,10,1.0)
img2 = cv2.warpAffine(img,rotation_matrix,(width, height))
```

```
img2_rgb = cv2.cvtColor(img2,cv2.COLOR_BGR2RGB)
gray2 = cv2.cvtColor(img2,cv2.COLOR_BGR2GRAY)
plt.figure(figsize=(8, 10))
plt.imshow(
    img2_rgb)
save_figure(
    "05_second_rotated_image.png",
    "Second Similar Image - Rotated 10 Degrees",
    figsize=(8, 10))
```




```
print("\n[STEP 8] Detecting SIFT features in both images...")
kp1, des1 = sift.detectAndCompute(gray,None)
kp2, des2 = sift.detectAndCompute(
    gray2,
    None
)print("\nImage 1 keypoints:", len(kp1))
print(
    "Image 2 keypoints:",
    len(kp2) )
if des1 is None or des2 is None:
```

```
    print("\nERROR: SIFT descriptors could not be generated.")
    sys.exit(1)
print("\n[STEP 9] Performing SIFT feature matching...")
bf = cv2.BFMatcher()
matches = bf.knnMatch(
    des1,
    des2,
    k=2 ) good_matches = []
for pair in matches:
    if len(pair) != 2:
        continue
        m, n = pair
 if m.distance < 0.75 * n.distance:
        good_matches.append(m)
print("\n" + "-" * 50)
print("SIFT MATCHING RESULTS")
print("-" * 50)
print("Total candidate matches:", len(matches))
print("Good matches:", len(good_matches))
print("\n[STEP 10] Creating SIFT matching visualization...")
match_image =
cv2.drawMatches(img,kp1,img2,kp2,good_matches,None,flags=cv2.DrawMatchesFlags_NOT_D
RAW_SINGLE_POINTS)
plt.figure(figsize=(18, 10))
plt.imshow(
    cv2.cvtColor(
        match_image,
        cv2.COLOR_BGR2RGB))
save_figure(
    "06_sift_feature_matching.png",
    "SIFT Feature Matching",
    figsize=(18, 10))


print("\n")
print("=" * 70)
print("SIFT vs HOG COMPARISON")
```

```
print("=" * 70)
print("""
+----------------------+----------------------+----------------------+
| Property             | SIFT                 | HOG                  |
+----------------------+----------------------+----------------------+
| Feature type         | Local keypoints      | Gradient features    |
| Main information     | Local texture        | Shape and edges      |
| Scale robustness     | Strong               | Limited              |
| Rotation robustness  | Strong               | Limited              |
| Image matching       | Excellent            | Not primary use      |
| Object detection     | Useful               | Very useful          |
| Feature location     | Sparse/local         | Dense/grid based     |
| Computation          | More complex         | Relatively simpler   |
+----------------------+----------------------+----------------------+
""")
print("\n")
print("=" * 70)
print("OBSERVATIONS")
print("=" * 70)
print("""
1. The input image was successfully loaded and processed.
2. The image was converted from RGB/color format to grayscale
   for feature extraction.
3. SIFT detected distinctive keypoints from important regions
   of the image.
4. SIFT descriptors were generated for the detected keypoints.
5. HOG extracted gradient orientation information from the image.
6. HOG represented important edge and shape information.
7. A second image was created by rotating the original image
   by 10 degrees.
8. SIFT feature matching successfully identified corresponding
   features between the two similar images.
9. SIFT is particularly useful for image matching and
   object recognition.
10. HOG is particularly useful for shape-based object detection
```

```
    and pedestrian detection.
11. Both techniques are handcrafted feature extraction methods
    and can be useful before traditional machine-learning
    classification or detection systems.
""")
print("\n")
print("=" * 70)
print("APPLICATIONS")
print("=" * 70)
print("""
SIFT Applications:
1. Image matching
2. Object recognition
3. Image stitching
4. Panorama generation
5. Object tracking
HOG Applications:
1. Pedestrian detection
2. Human detection
3. Object detection
4. Shape recognition
5. Computer vision classification
""")
print("\n[STEP 14] Saving feature information...")
if descriptors is not None:
    np.save(
        os.path.join(
            OUTPUT_DIR,
            "sift_descriptors.npy"),
        descriptors)
np.save(
    os.path.join(
        OUTPUT_DIR,
        "hog_features.npy"),
    hog_features
```

```
)
print("\n")
print("=" * 70)
print("EXPERIMENT COMPLETED SUCCESSFULLY")
print("=" * 70)
print("""
Generated files:
output/
├── 01_original_image.png
├── 02_grayscale_image.png
├── 03_sift_keypoints.png
├── 04_hog_visualization.png
├── 05_second_rotated_image.png
├── 06_sift_feature_matching.png
├── sift_descriptors.npy
└── hog_features.npy
""")
print("=" * 70)
print("You can use the six PNG files as screenshots/results")
print("for your practical record.")
print("=" * 70)
```

QUESTIONS & ANSWERS

---

1. What is feature extraction, and why is it important in computer vision?

Definition:
Feature extraction is the process of identifying and representing important characteristics or patterns from an image, such as **edges, shapes, textures, corners, and gradients.**

Importance:
It converts a complex image into useful numerical features that can be easily processed by computer vision algorithms. It reduces unnecessary information and helps in **classification, object detection, and image matching.**

Example:
SIFT extracts distinctive keypoints from an image, while HOG extracts edge and shape information.

---

2. Explain the working principle of Scale-Invariant Feature Transform (SIFT).

Definition:
SIFT is a feature extraction technique used to detect and describe distinctive features in an image.

Working:

1. Detects important **keypoints** at different scales.
2. Assigns an orientation to the keypoints.
3. Generates a **128-dimensional descriptor** for each keypoint.
4. These descriptors can be compared to find matching features between images.

Example:
SIFT can match the same object in two images even if one image is **rotated or resized**.

---

3. What are keypoints and feature descriptors in image analysis?

Keypoints:
Keypoints are distinctive points or regions in an image that contain important visual information, such as **corners, edges, or textured areas.**

Feature Descriptors:
A feature descriptor is a numerical representation of the local region around a keypoint. It allows the computer to compare features between images.

Example:
In SIFT, detected corners or distinctive regions are **keypoints**, and each keypoint is represented by a **128-dimensional descriptor**.

---

4. Explain the concept of Histogram of Oriented Gradients (HOG) and its significance.

Definition:
HOG is a feature descriptor that represents an image using the **direction and magnitude of gradients** in local regions.

Working:
The image is divided into small **cells**. Gradient directions are calculated and placed into orientation histograms. These histograms are normalized using blocks.

Significance:
HOG is effective for representing **shape and edge information**, making it useful for object detection.

Example:
HOG is commonly used for **pedestrian detection**.

---

5. Compare SIFT and HOG based on robustness, computational complexity, and practical applications.

Feature	SIFT	HOG
Main feature	Local keypoints and texture	Gradients, edges and shape
Scale robustness	Strong	Limited
Rotation robustness	Strong	Limited
Complexity	More computationally complex	Relatively simpler
Main application	Image matching, object recognition	Object/pedestrian detection

Example:
SIFT can match a rotated object, whereas HOG is commonly used to detect the shape of a person in an image.

---

6. Why is SIFT considered invariant to scale and rotation?

Answer:
SIFT is **scale invariant** because it detects features at multiple image scales using a scale-space representation. Therefore, the same feature can be detected even when the image is resized.

It is **rotation invariant** because SIFT assigns a dominant orientation to each keypoint and creates the descriptor relative to that orientation.

Example:
A logo can be recognized by SIFT even when it appears **larger**, **smaller**, **or rotated** in another image.

---

7. Mention three real-world applications where HOG descriptors are commonly used.

Three applications of HOG are:

1. **Pedestrian detection** in surveillance and autonomous vehicles.
2. **Human detection** in images and videos.
3. **Object detection** based on shape and edge information.

Example:
A security camera can use HOG-based methods to detect people entering a restricted area.

---

8. Why is feature extraction performed before image classification or object detection?

Answer:
Feature extraction is performed to convert raw image data into **meaningful and compact numerical information.**

It helps the classifier focus on important characteristics such as **edges**, **shapes**, **textures**, **and patterns** instead of processing every pixel directly.

Example:
HOG features can represent the shape of a person before a classifier determines whether the image contains a **pedestrian.**

---

9. What are the advantages and limitations of handcrafted feature descriptors compared to deep learning-based feature extraction?

Advantages of handcrafted features:

- Simple and interpretable.
- Require less computational power.
- Can work well with limited training data.
- Examples include **SIFT and HOG.**

Limitations:

- Designed using manually selected rules.
- May not perform well on complex image variations.
- Usually less adaptable than deep learning features.

Example:
SIFT and HOG can work effectively for specific tasks, while **CNNs** can automatically learn complex features from large datasets.

---

10. How do feature extraction techniques contribute to image matching, face recognition, and object detection systems?

Answer:
Feature extraction converts images into useful numerical representations that can be compared or classified.

- **Image matching:** SIFT descriptors are compared to find corresponding features.
- **Face recognition:** Facial features such as edges, shapes, and textures can be extracted and compared.
- **Object detection:** HOG can represent shape and edge information to detect objects.

Example:
SIFT can identify matching points between two images of the same object, while HOG can help detect a **person based on their shape.**
